

Reviewer Pack — Tropical Analytic Rank and Resolution Proof Width Inequality

Entry 8616e6c721a4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 04:19:38 UTC

Tropical Analytic Rank and Resolution Proof Width Inequality

Entry ID: 8616e6c721a4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 04:19:38 UTC

1. Conjecture

Field A (mathematical branch): Tropical Analysis (Analytic Number Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every d -regular graph G , the tropical analytic rank ($\text{tar}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\text{tar}(G) \leq cw(\varphi_G)$ for some constant $c > 0$.

Rationale (proposer's reasoning):

The tropical analytic rank provides a measure of complexity in the max-plus semiring, which may expose subtle properties of formulas. This conjecture suggests that it could serve as a lower bound on resolution proof width, offering a new perspective on proof complexity.

Taxonomy category: TROPICAL_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 69bd56807875db2f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each graph G , if the ratio of tropical analytic rank to resolution proof width is less than or equal to a constant c within 95% confidence interval and no seed produces a ratio greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical analysis AND resolution proof complexity - analytic rank tropical AND resolution proof width inequality - Tseitin formula AND tropical analytic rank correlated with resolution proof width

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/2111.10328v3>] The Four Point Condition: An Elementary Tropicalization of Ptolemy's Inequality - [<http://arxiv.org/abs/1511.006>] A specialization inequality for tropical complexes - [<http://arxiv.org/abs/1001.1554v4>] Tropical geometry and correspondence theorems via toric stacks - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(b[i], A[i][i])
        for j in range(i-1, -1, -1):
            b[j] -= A[j][i] * x[i]
    return x

def matrix_multiply(A, B):
    n = len(A)
```

```

C = [[0]*n for _ in range(n)]
for i in range(n):
    for j in range(n):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(d, n):
        if (n * d) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges = set()
        while len(edges) < n * d // 2:
            u = random.randint(0, n-1)
            v = random.randint(0, n-1)
            if u == v or (u, v) in edges or (v, u) in edges:
                continue
            graph[u].append(v)
            graph[v].append(u)
            edges.add((u, v))
        return graph

    def tseitin_formula(graph):
        n = len(graph)
        literals = [random.randint(1, 2*n) for _ in range(n)]
        clauses = []
        for i in range(n):
            clauses.append([literals[i]])
        for u in range(n):
            for v in graph[u]:
                if u < v:
                    new_lit = random.randint(1, 2*n)
                    clauses.append([-new_lit, literals[u], literals[v]])
                    clauses.append([-new_lit, -literals[u], -literals[v]])
                    clauses.append([new_lit])
        return clauses

    def tropical_analytic_rank(clauses):
        n = len(clauses)
        A = [[0]*n for _ in range(n)]
        b = [0]*n
        for i in range(n):
            for j in range(i+1, n):

```

```

        if any(lit in clauses[i] and -lit in clauses[j] for lit in literals):
            A[i][j] = 1
            A[j][i] = 1
    det = determinant(A)
    return abs(det) if det != 0 else float('inf')

def resolution_width(clauses):
    n = len(clauses)
    clauses = [set(clause) for clause in clauses]
    queue = list(clauses)
    while queue:
        clause = queue.pop()
        if not clause:
            return 1
        lits = list(clause)
        other_lit = random.choice(lits)
        new_clauses = []
        for c in queue:
            if other_lit in c:
                continue
            if -other_lit in c:
                new_clauses.append(c - {other_lit})
            else:
                new_clauses.append(c | {-other_lit})
        queue.extend(new_clauses)
    return float('inf')

n = random.choice([10, 20, 30, 40])
d = random.randint(2, min(n-1, 5))
graph = generate_d_regular_graph(d, n)
if not graph:
    return {
        "metric_name": "tropical_analytic_rank",
        "metric_value": float('inf'),
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
clauses = tseitin_formula(graph)
tar = tropical_analytic_rank(clauses)
w = resolution_width(clauses)

if tar == float('inf') or w == float('inf'):
    return {
        "metric_name": "tropical_analytic_rank",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

return {
    "metric_name": "tropical_analytic_rank",
    "metric_value": tar / w,

```

```

        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        max_ratio = max(r["metric_value"] for r in results if r["conjecture_holds"])
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif max_ratio > 10:
        print(f"RESULT: FALSIFIED counterexample=\"max_ratio_exceeds_10\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(seeds)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05a86ac5.py", line 164, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05a86ac5.py", line 137, in run_trial
    tar = tropical_analytic_rank(clauses)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_05a86ac5.py", line 97, in tropical_analytic_rank
    if any(lit in clauses[i] and -lit in clauses[j] for lit in literals):
                                   ~~~~~^
NameError: name 'literals' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19807
2	preregistration	ollama_remote	glm4:latest	0	0	8942
3	novelty	ollama_remote	glm4:latest	0	0	8259
4	novelty	ollama_remote	glm4:latest	0	0	24027
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22618
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30319
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21059
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26128
9	judge	ollama_remote	glm4:latest	0	0	8772

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169931 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8616e6c721a4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8616e6c721a4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8616e6c721a4.tar.gz` (if generated)